Лекция 13: Рекурсивные алгоритмы. Простые сортировки

Введение

В этой лекции мы рассмотрим два фундаментальных понятия, которые пронизывают всю дисциплину «Алгоритмы и структуры данных»:

Рекурсия - мощный приём, позволяющий описывать решение задачи через неё саму, но на меньшем размере входных данных.

Простые сортировки - базовые алгоритмы упорядочивания, на которых удобно отрабатывать навыки анализа сложности, устойчивости и адаптивности.

В начале файла необходимо подключить следующие заголовки:

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <functional>
using namespace std;
```

Вопрос 1. Рекурсивные алгоритмы

1.1. Что такое рекурсия

Рекурсия - это способ определения функции (или алгоритма), при котором функция вызывает саму себя с изменённым (как правило, уменьшенным) набором аргументов.

Классический пример - вычисление факториала:

$$n! = n \cdot (n-1)!, \text{ при } n > 0$$

$$0! = 1$$

Здесь n! выражается через (n-1)! - то же самое понятие, но для меньшего числа.

Важно (C++): рекурсивная функция должна быть объявлена до её вызова либо иметь прототип. При косвенной рекурсии обычно используют **предварительные объявления**.

1.2. Прямая и косвенная рекурсия

Прямая рекурсия - функция вызывает саму себя непосредственно.

```
long long factorial(int n) {
   if (n == 0) return 1;         // базовый случай
               return n * factorial(n - 1);   // прямой вызов себя
```

Косвенная рекурсия - функция A вызывает функцию B, а та, в свою очередь, вызывает A (цикл из двух и более функций). В C++ для этого нужны предварительные объявления:

```
bool isEven(int n);   // предварительное объявление
bool isOdd(int n);    // предварительное объявление

bool isEven(int n) {
   if (n == 0) return true;
   return isOdd(n - 1);       // косвенный вызов
}

bool isOdd(int n) {
   if (n == 0) return false;
   return isEven(n - 1);      // косвенный вызов
            }
```

Здесь is_even и is_odd образуют косвенную рекурсию. Такой приём полезен, когда задача естественно распадается на чередующиеся подзадачи.

1.3. Базовый случай и шаг рекурсии

Любая корректная рекурсивная функция обязана содержать две части:

Компонент	Назначение
Базовый случай (base case)	Условие остановки. Без него рекурсия бесконечна (приведёт к переполнению стека).
Шаг рекурсии (recursive step)	Сведение задачи к подзадаче меньшего размера + объединение результатов.

Пример - сумма элементов массива:

```
long long sumArray(const vector<int>& a, size_t i = 0) {
   if (i == a.size()) return 0;              // базовый случай

   return a[i] + sumArray(a, i + 1);         // шаг рекурсии
           }
```

Правила проектирования рекурсии:

1. Определить базовый случай (часто несколько).
2. Убедиться, что каждый рекурсивный вызов приближает к базовому случаю.
3. Проверить, что комбинация результатов подзадач даёт результат исходной задачи.

Замечание о глубине стека. По умолчанию в типичных реализациях C++ стек выдерживает порядка 10^4–10^5 вложенных вызовов. Для больших n применяют итеративный подход либо явный стек (std::stack).

1.4. «Явный стек» и «неявный стек»

«Явный стек» означает стек, который вы создаёте вручную в коде (обычно через std::stack или std::vector), вместо того чтобы полагаться на неявный стек вызовов, который автоматически использует процессор и операционная система при рекурсии.

Разница между двумя стеками

1. Неявный стек (стек вызовов / call stack)

Это память, которой управляет сам процессор и ОС. Каждый рекурсивный вызов функции кладёт на него новый фрейм (локальные переменные, адрес возврата, аргументы).

```
void dfs(int v) {
   visited[v] = true;
   for (int u : adj[v])
      if (!visited[u])
         dfs(u);   // каждый вызов → новый фрейм на стеке вызовов
           }
```

Размер ограничен (обычно 1–8 МБ).

При глубокой рекурсии (~10^4–10^5 вызовов) → stack overflow.

Вы не контролируете его размер.

Явный стек (explicit stack)

Вы сами создаёте структуру данных (std::stack, std::vector) в куче (heap), где памяти гораздо больше, и вручную эмулируете рекурсию.

```
//Функция обхода, начиная с вершины start.
void dfs(int start) {
   std::stack<int> st;      // явный стек в куче; Создаём пустой стек и кладём туда стартовую вершину. Стек хранит вершины, которые ещё предстоит обработать.
   st.push(start);

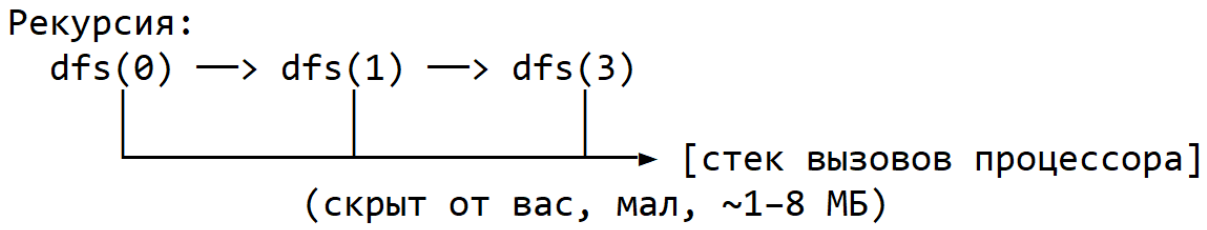
   //Крутимся, пока есть непосещённые вершины в стеке.
   while (!st.empty()) {
      int v = st.top();//Берём вершину с вершины стека и сразу убираем её. v — текущая вершина для обработки.
      st.pop();

      if (visited[v]) continue;//Проверка на дубликат. Вершина могла попасть в стек несколько раз (если у неё несколько соседей-предшественников).
      visited[v] = true;   // Если уже посещена — пропускаем. Иначе помечаем как посещённую.

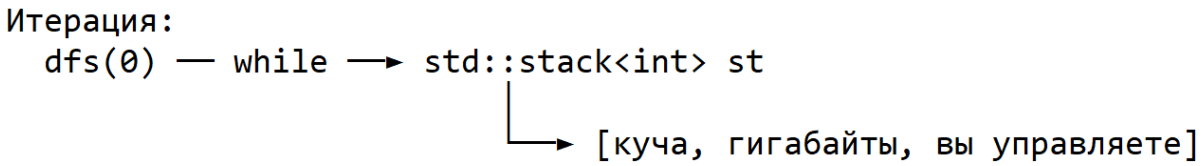
      //Проходим по всем соседям u вершины v.
      //Если сосед ещё не посещён — кладём его в стек.
      // Это аналог рекурсивного вызова dfs(u) : вместо того чтобы «провалиться» в него сразу,
      //  мы откладываем его на потом, положив в стек.

      for (int u : adj[v])
         if (!visited[u])
            st.push(u);  // "замена" рекурсивного вызова
   }
   //Когда стек опустеет — все достижимые вершины обработаны.
}
```

Когда программа выполняет рекурсию, «отложенные дела» (к кому вернуться, какие переменные сохранить) хранит **сам процессор** - в специальной области памяти. Вы этого не видите и не контролируете.



Явный стек - вы **сами** создаёте такую структуру в коде:



Явный стек - это когда вы вручную ведёте список «что ещё осталось обработать», вместо того чтобы позволить процессору вести его за вас через рекурсию.

```
std::stack<int> st;   // ← вот он, явный стек. Виден. Ваш.
st.push(u);           // ← сами кладёте
int v = st.top();     // ← сами берёте
st.pop();             // ← сами убираете
```

В рекурсивной версии ничего этого нет - стек **неявный**, спрятан внутри механизма вызовов функций:

```
dfs(u);   // где хранится "отложенное"? Не видно. Это неявный стек.
```

	Неявный (рекурсия)	**Явный (std::stack)**
Пишете код	Короче, элегантнее	Длиннее, вручную
Контроль	Нет	Полный
Риск переполнения	Есть (глубина ~10^4–10^5)	Практически нет
Где живёт	Стек вызовов (мал)	Куча (велика)

1. Рекурсия = «отложенные дела» хранит процессор в неявном стеке.
2. Явный стек = вы сами храните эти дела в std::stack.
3. Смысл: обойти ограничение на глубину и получить контроль над памятью.

Размер ограничен только доступной памятью (гигабайты).

1. Нет риска переполнения стека вызовов.
2. Работает при любой глубине n.

Почему это называют «явным»

	Неявный стек	**Явный стек**
Кто управляет	Процессор/ОС	Программист
Где живёт	Стек вызовов	Куча (std::stack)
Виден в коде	Нет (скрыт в рекурсии)	Да (вы его пишете)
Размер	Маленький, фиксированный	Большой, динамический

Слово «явный» подчёркивает: стек теперь явно выражен в коде как объект std::stack, а не спрятан внутри механизма вызовов функций.

Когда применяют

1. Обход графов/деревьев (DFS) при больших n.
2. Динамическое программирование с глубокой рекурсией.
3. Парсинг вложенных структур.
4. Любая задача, где рекурсия упирается в лимит стека.

Явный стек = вы вручную моделируете рекурсию, используя std::stack в куче, чтобы обойти ограничение размера стека вызовов и избежать stack overflow при больших n.

1.5. Рекурсивный обход односвязанного списка

Список (односвязный) - рекурсивная структура по своей природе: «голова + хвост». В C++ удобно использовать «сырые» указатели или std::unique_ptr.

Односвязный список рекурсивен по своей природе, потому что каждый его узел содержит **голову** (значение) и **хвост** - ссылку на **список** из

оставшихся элементов. Определение списка ссылается на само себя, а точка остановки - пустой список (nullptr). Именно поэтому рекурсивные функции над списком выглядят так естественно: они буквально повторяют форму данных.

Вариант с сырыми указателями:

```
struct Node {
   int value;
   Node* next;
   Node(int v, Node* n = nullptr) : value(v), next(n) {}
};

void printList(const Node* node) {
   if (node == nullptr) return;    // базовый случай: конец списка
   cout << node->value << ' ';
   printList(node->next);          // шаг: обход хвоста
            }
```

Порядок обработки можно менять - обход «от хвоста к голове» получается, если печатать значение **после** рекурсивного вызова.

Вариант с std::unique_ptr (современный C++):

```
// ──────────────────────────────────────────────────────────────────────────
// Структура узла односвязного списка с использованием unique_ptr.
// unique_ptr — "умный указатель": сам освобождает память,
// когда узел больше не нужен. Не нужно вручную вызывать delete.
// ──────────────────────────────────────────────────────────────────────────
struct NodeUP {
   int value;                      // значение, которое хранит узел
   unique_ptr<NodeUP> next;        // умный указатель на следующий узел
   // (или nullptr, если это последний узел)

// ─── Конструктор ──────────────────────────────────────────────────────────
// Вызывается при создании узла: NodeUP(5) или NodeUP(5, std::move(ptr))
//
//   v — значение, которое кладём в узел
//   n — указатель на следующий узел (по умолчанию nullptr,
//       то есть узел создаётся как последний в списке)
//
// : value(v), next(std::move(n)) — список инициализации:
//   value сразу получает v, а next "забирает" владение у n.
//
// std::move нужен потому, что unique_ptr НЕЛЬЗЯ копировать
// (он единственный владелец памяти), но МОЖНО перемещать —
// то есть передавать владение от n к next.
// ──────────────────────────────────────────────────────────────────────────
   NodeUP(int v, unique_ptr<NodeUP> n = nullptr)
      : value(v), next(std::move(n)) {}
};


// ──────────────────────────────────────────────────────────────────────────
// Рекурсивная печать списка.
// node — указатель на текущий узел (обычный "наблюдающий" указатель,
//        мы НЕ владеем памятью, только смотрим на неё).
// ──────────────────────────────────────────────────────────────────────────
void printListUP(const NodeUP* node) {
```

```
    // ——— Базовый случай рекурсии ————————————————————————————
    // Если node == nullptr, значит мы дошли до конца списка —
    // печатать нечего, выходим из функции.
    // (Рекурсия без базового случая = бесконечный цикл и крах.)
    if (!node) return;

    // ——— Печатаем "голову" ——————————————————————————————————
    // node->value — берём значение из узла, на который указывает node.
    // ' ' — пробел после числа для читаемости.
    cout << node->value << ' ';

    // ——— Рекурсивно печатаем "хвост" ————————————————————————
    // node->next — это unique_ptr на следующий узел.
    // .get() возвращает ОБЫЧНЫЙ указатель (NodeUP*) из unique_ptr,
    //        не передавая владение. Именно его ждёт функция
    //        (она принимает const NodeUP*, а не unique_ptr).
    //
    // Так мы "откусываем голову" и повторяем ту же процедуру
    // для оставшейся части списка — она тоже список.
    printListUP(node->next.get());
        }
```

1.6. Рекурсивный обход дерева

Дерево - ещё более «рекурсивная» структура: каждое поддерево само является деревом.

```
#include <iostream>

using namespace std;

// ————————————————————————————————————————————————————————————————
// СТРУКТУРА УЗЛА БИНАРНОГО ДЕРЕВА
//
// Дерево — это рекурсивная структура: каждый узел хранит
// значение и ДВА указателя на поддеревья (левое и правое).
// Каждое поддерево — снова дерево, устроенное точно так же.
//
//          (value)
//          /     \
//      left      right
//     (поддерево) (поддерево)
//
// Если у узла нет левого или правого ребёнка — там nullptr.
// ————————————————————————————————————————————————————————————————
struct TreeNode {

    int value;          // значение, которое хранит узел

    TreeNode* left;     // указатель на левое поддерево
    // (nullptr, если левого ребёнка нет)

    TreeNode* right;    // указатель на правое поддерево
    // (nullptr, если правого ребёнка нет)

// ——— КОНСТРУКТОР ————————————————————————————————————————————
// Вызывается при создании узла.
//
//   v — значение для узла
//   l — указатель на левое поддерево  (по умолчанию nullptr)
//   r — указатель на правое поддерево (по умолчанию nullptr)
//
// После двоеточия — СПИСОК ИНИЦИАЛИЗАЦИИ:
// просто кладём переданные значения в поля структуры.
// ————————————————————————————————————————————————————————————————
```

```
    TreeNode(int v, TreeNode* l = nullptr, TreeNode* r = nullptr)
        : value(v), left(l), right(r) {}
};


// ═══════════════════════════════════════════════════════════════════════
// ОБХОД №1: ПРЯМОЙ ПОРЯДОК (preorder)
// Порядок: КОРЕНЬ → ЛЕВОЕ → ПРАВОЕ
//
// Сначала печатаем сам узел, потом идём в левое поддерево,
// затем в правое. Корень всегда идёт первым.
//
// node — ОБЫЧНЫЙ указатель (не владеющий). Функция только
// смотрит на дерево и не удаляет узлы.
// const означает: "я обещаю не менять узлы".
// ═══════════════════════════════════════════════════════════════════════
void preorder(const TreeNode* node) {

    // ─── БАЗОВЫЙ СЛУЧАЙ ──────────────────────────────────────────────
    // Если node == nullptr — мы дошли до пустого места,
    // идти дальше некуда. Выходим.
    // Без этого условия рекурсия была бы бесконечной.
    if (!node) return;

    // ─── 1. КОРЕНЬ ───────────────────────────────────────────────────
    // Сначала печатаем значение текущего узла.
    cout << node->value << ' ';

    // ─── 2. ЛЕВОЕ ────────────────────────────────────────────────────
    // Рекурсивно обходим левое поддерево тем же правилом.
    preorder(node->left);

    // ─── 3. ПРАВОЕ ───────────────────────────────────────────────────
    // Затем — правое поддерево.
    preorder(node->right);
}


// ═══════════════════════════════════════════════════════════════════════
// ОБХОД №2: СИММЕТРИЧНЫЙ ПОРЯДОК (inorder)
// Порядок: ЛЕВОЕ → КОРЕНЬ → ПРАВОЕ
//
// Сначала полностью обходим левое поддерево, потом печатаем
// узел, потом правое поддерево. Корень идёт В СЕРЕДИНЕ.
//
// Важно: для бинарного ДЕРЕВА ПОИСКА (BST) этот обход печатает
// значения В ОТСОРТИРОВАННОМ ПОРЯДКЕ — это его главное свойство.
// ═══════════════════════════════════════════════════════════════════════
void inorder(const TreeNode* node) {

    // ─── БАЗОВЫЙ СЛУЧАЙ ──────────────────────────────────────────────
    if (!node) return;

    // ─── 1. ЛЕВОЕ ────────────────────────────────────────────────────
    // Сначала целиком разбираемся с левым поддеревом.
    inorder(node->left);

    // ─── 2. КОРЕНЬ ───────────────────────────────────────────────────
    // Только теперь печатаем сам узел.
    cout << node->value << ' ';

    // ─── 3. ПРАВОЕ ───────────────────────────────────────────────────
    inorder(node->right);
}
```

```
// ═══════════════════════════════════════════════════════════════════════
// ОБХОД №3: ОБРАТНЫЙ ПОРЯДОК (postorder)
// Порядок: ЛЕВОЕ → ПРАВОЕ → КОРЕНЬ
//
// Сначала обходим оба поддерева, и только потом печатаем
// сам узел. Корень идёт ПОСЛЕДНИМ.
//
// Применяется, когда нужно сначала обработать детей, а потом
// родителя: удаление дерева, вычисление размеров, разбор
// выражений и т.п.
// ═══════════════════════════════════════════════════════════════════════
void postorder(const TreeNode* node) {

    // ─── БАЗОВЫЙ СЛУЧАЙ ──────────────────────────────────────────────
    if (!node) return;

    // ─── 1. ЛЕВОЕ ────────────────────────────────────────────────────
    postorder(node->left);

    // ─── 2. ПРАВОЕ ───────────────────────────────────────────────────
    postorder(node->right);

    // ─── 3. КОРЕНЬ ───────────────────────────────────────────────────
    // Печатаем узел ТОЛЬКО после того, как обошли обоих детей.
    cout << node->value << ' ';
        }
```

Как запомнить три порядка

Всё отличие - **в каком месте стоит печать узла** относительно двух рекурсивных вызовов:

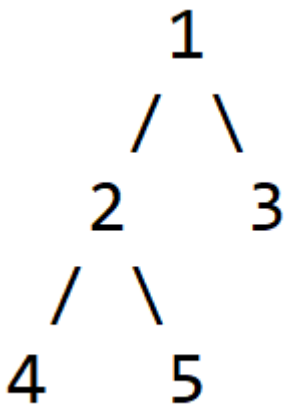
```
preorder:   cout << value;  left();  right();   // корень в НАЧАЛЕ
inorder:    left();  cout << value;  right();   // корень В СЕРЕДИНЕ
postorder:  left();  right();  cout << value;   // корень В КОНЦЕ
```

Название говорит, когда обрабатывается корень:

1. preorder - корень до детей.
2. inorder - корень между детьми.
3. postorder - корень после детей.

Пример: что печатает каждый обход

Возьмём дерево:



Обход	Порядок печати
preorder	1 2 4 5 3
inorder	4 2 5 1 3
postorder	4 5 2 3 1

```
int main() {
    // Строим дерево снизу вверх:
    //       1
    //      / \
    //     2   3
    //    / \
    //   4   5
    auto n4 = new TreeNode(4);
    auto n5 = new TreeNode(5);
    auto n2 = new TreeNode(2, n4, n5);
    auto n3 = new TreeNode(3);
    auto n1 = new TreeNode(1, n2, n3);

    cout << "preorder:  "; preorder(n1);   cout << endl;
    cout << "inorder:   "; inorder(n1);    cout << endl;
    cout << "postorder: "; postorder(n1);  cout << endl;

    deleteTree(n1); // Не забыть удалить дерево!

            }
```

Все три обхода - один и тот же алгоритм, отличаются лишь позицией cout. Это прямое следствие рекурсивной природы дерева:

Дерево = корень + левое поддерево + правое поддерево, где каждое поддерево - снова дерево.

Каждая функция просто повторяет эту формулу, а порядок обхода задаётся тем, в какой момент мы «посещаем» корень.

1.7. Ханойские башни

Задача. Даны три стержня (A, B, C) и n дисков разного диаметра на стержне A. Нужно перенести все диски на стержень C, соблюдая правила:

1. за один ход переносится один диск;
2. больший диск нельзя класть на меньший.

https://yandex.ru/alice?share=e2f0d56d-876c-f4e8-2621-1e26811a398d

Рекурсивное решение. Чтобы перенести n дисков с A на C через вспомогательный B:

1. Перенести n-1 диск с A на B (используя C как вспомогательный).
2. Перенести самый большой диск с A на C.
3. Перенести n-1 диск с B на C (используя A как вспомогательный).

```
void hanoi(int n, char source, char target, char auxiliary) {
   if (n == 1) {                          // базовый случай
      cout << source << " -> " << target << '\n';
      return;
   }
   hanoi(n - 1, source, auxiliary, target);       // шаг 1
   cout << source << " -> " << target << '\n';    // шаг 2
   hanoi(n - 1, auxiliary, target, source);       // шаг 3
         }
```

Подсчёт ходов. Обозначим T(n) - число ходов для n дисков.

$T(1) = 1$

$T(n) = 2 \cdot T(n-1) + 1$

Разворачивая рекуррентность:

$T(n) = 2 \cdot T(n-1) + 1$

$= 2^2 \cdot T(n-2) + 2 + 1$

$= ...$

$= 2^{n-1} \cdot T(1) + 2^{n-2} + ... + 2 + 1 \quad = 2^n - 1$

Итак, $\mathbf{T(n) = 2^n - 1}$. Для 64 дисков это $\approx 1.8 \cdot 10^{19}$ ходов - при 1 ходе в секунду потребуется более 500 миллиардов лет. Это иллюстрирует экспоненциальную сложность.

```
#include <iostream>

using namespace std;


// ═══════════════════════════════════════════════════════════════════════
// РЕКУРСИВНАЯ ФУНКЦИЯ ХАНОЙСКОЙ БАШНИ
//
// Параметры:
//   n         — сколько дисков нужно перенести
//   source    — откуда переносим
//   target    — куда переносим
//   auxiliary — вспомогательный стержень (помощник)
//
// Функция печатает последовательность ходов.
// ═══════════════════════════════════════════════════════════════════════
void hanoi(int n, char source, char target, char auxiliary) {

    // ─── БАЗОВЫЙ СЛУЧАЙ ──────────────────────────────────────────────
    // Если остался всего 1 диск — просто переносим его
    // напрямую с source на target и выходим.
    // Без этого условия рекурсия была бы бесконечной.
    if (n == 1) {
        cout << source << " -> " << target << '\n';
        return;
    }

    // ─── ШАГ 1 ──────────────────────────────────────────────────────
    // Переносим n-1 диск с source на auxiliary,
    // используя target как вспомогательный стержень.
    //
    // Внимание: здесь target и auxiliary МЕНЯЮТСЯ МЕСТАМИ
    // в аргументах! Мы "освобождаем" target для самого
    // большого диска.
    hanoi(n - 1, source, auxiliary, target);

    // ─── ШАГ 2 ──────────────────────────────────────────────────────
    // Теперь на source остался только самый большой диск.
    // Переносим его напрямую на target — это и есть один ход.
    cout << source << " -> " << target << '\n';

    // ─── ШАГ 3 ──────────────────────────────────────────────────────
    // Переносим n-1 диск с auxiliary на target,
    // используя source как вспомогательный стержень.
    //
    // Здесь source и auxiliary меняются местами:
    // source уже пуст, поэтому он теперь "помощник".
    hanoi(n - 1, auxiliary, target, source);
}


// ═══════════════════════════════════════════════════════════════════════
// ГЛАВНАЯ ФУНКЦИЯ
// ═══════════════════════════════════════════════════════════════════════
int main() {
    int n;
```

```
    // Просим пользователя ввести число дисков.
    cout << "Введите число дисков: ";
    cin >> n;

    // Проверка на корректность: n должно быть >= 1.
    if (n < 1) {
        cout << "Число дисков должно быть положительным.\n";
        return 1;
    }

    // Сообщаем, что начинаем.
    cout << "\nХоды для " << n << " дисков:\n";
    cout << "--------------------\n";

    // Стержни называем A, B, C.
    // Переносим n дисков с A на C через B.
    hanoi(n, 'A', 'C', 'B');

    // Считаем и выводим общее число ходов.
    // Известно, что минимальное число ходов = 2^n - 1.
    // (1 << n) — это побитовый сдвиг, эквивалент 2^n.
    cout << "--------------------\n";
    cout << "Всего ходов: " << ((1 << n) - 1) << '\n';

    return 0;
            }
```

1.8. Обход лабиринта (backtracking)

Backtracking (метод проб и возврата) - это рекурсивная стратегия перебора вариантов: пробуем ход, рекурсивно идём дальше; если тупик - возвращаемся и пробуем другой вариант.

```
// ═══════════════════════════════════════════════════════════════════════
// ПОИСК ПУТИ В ЛАБИРИНТЕ (backtracking / перебор с возвратом)
//
// Лабиринт — двумерная сетка:
//   0 — свободная клетка (можно идти)
//   1 — стена (идти нельзя)
//
// Нужно найти путь из левого верхнего угла (0,0)
// в правый нижний (rows-1, cols-1).
//
// Идея: пробуем идти во все стороны по очереди. Если из клетки
// удалось дойти до финиша — возвращаем true. Если нет — откатываем
// последний шаг (возвращаемся назад) и пробуем другое направление.
// ═══════════════════════════════════════════════════════════════════════

#include <iostream>
#include <vector>

using namespace std;


// ═══════════════════════════════════════════════════════════════════════
// РЕКУРСИВНАЯ ФУНКЦИЯ ПОИСКА ПУТИ
//
// Параметры:
//   maze    — сам лабиринт (0 — свободно, 1 — стена)
//   x, y    — текущая клетка, в которой мы находимся
//   path    — накапливаемый путь: список посещённых клеток
//             (если дойдём до финиша — здесь будет ответ)
```

```
//   visited — какие клетки мы уже посетили на текущем шаге
//
// Возвращает true, если путь найден, и false — если нет.
//
// maze и visited передаём ПО ССЫЛКЕ (знак &), чтобы не копировать
// их на каждом вызове — это было бы очень медленно.
// ──────────────────────────────────────────────────────────────────────────────
bool solveMaze(vector<vector<int>>& maze,
   int x, int y,
   vector<pair<int, int>>& path,
   vector<vector<bool>>& visited)
{
   // Узнаём размеры лабиринта.
   int rows = maze.size();       // количество строк
   int cols = maze[0].size();    // количество столбцов

   // ─── ПРОВЕРКА: МОЖНО ЛИ ВООБЩЕ СТОЯТЬ В ЭТОЙ КЛЕТКЕ ──────
   // Если вышли за границы лабиринта — туда нельзя.
   if (x < 0 || y < 0 || x >= rows || y >= cols) return false;

   // Если это стена (maze[x][y] == 1) — туда нельзя.
   // Если мы уже были здесь на текущем шаге (visited[x][y]) —
   // тоже нельзя, иначе зациклимся.
   if (maze[x][y] == 1 || visited[x][y]) return false;

   // ─── ОТМЕЧАЕМ КЛЕТКУ КАК ПОСЕЩЁННУЮ ──────────────────────
   // Раз мы дошли сюда и не отсеялись — запоминаем это.
   visited[x][y] = true;

   // И добавляем клетку в текущий путь.
   // path.push_back({x, y}) кладёт пару (x, y) в конец списка.
   path.push_back({ x, y });

   // ─── БАЗОВЫЙ СЛУЧАЙ: ДОШЛИ ДО ФИНИША ─────────────────────
   // Правый нижний угол — это цель. Если мы здесь — путь найден.
   // Возвращаем true, и все предыдущие вызовы тоже вернут true,
   // "протаскивая" успех наверх по цепочке рекурсии.
   if (x == rows - 1 && y == cols - 1) return true;

   // ─── ПРОБУЕМ 4 НАПРАВЛЕНИЯ ──────────────────────────────
   // dx и dy — смещения для четырёх соседних клеток:
   //   вверх  (x-1, y)
   //   вниз   (x+1, y)
   //   влево  (x, y-1)
   //   вправо (x, y+1)
   const int dx[4] = { -1, 1, 0, 0 };
   const int dy[4] = { 0, 0, -1, 1 };

   // Перебираем все четыре направления.
   for (int k = 0; k < 4; ++k) {

      // Пытаемся шагнуть в соседнюю клетку.
      // Если рекурсивный вызов вернёт true — значит,
      // из этой клетки удалось дойти до финиша.
      if (solveMaze(maze, x + dx[k], y + dy[k], path, visited))
         return true;   // УРА! Путь найден — возвращаем true наверх.
   }

   // ─── ОТКАТ (BACKTRACKING) ────────────────────────────────
   // Если ни одно из четырёх направлений не привело к финишу,
   // значит эта клетка — тупик. Надо "откатить" изменения:
   //
   //   1) убрать её из текущего пути (path.pop_back())
   //   2) снять отметку "посещена" (visited[x][y] = false),
   //      чтобы другие ветки поиска могли снова сюда зайти.
   //
```

```
    // После отката возвращаем false — "отсюда пути нет".
    path.pop_back();
    visited[x][y] = false;
    return false;
}


// ──────────────────────────────────────────────────────────────
// ГЛАВНАЯ ФУНКЦИЯ — пример использования
// ──────────────────────────────────────────────────────────────
int main() {
    // Лабиринт 5x5.
    // 0 — свободно, 1 — стена.
    vector<vector<int>> maze = {
        {0, 0, 0, 0, 1},
        {1, 1, 0, 1, 0},
        {0, 0, 0, 0, 0},
        {0, 1, 1, 1, 0},
        {0, 0, 0, 1, 0}
    };

    int rows = maze.size();
    int cols = maze[0].size();

    // Путь и отметки посещений — пустые в начале.
    vector<pair<int, int>> path;
    vector<vector<bool>> visited(rows, vector<bool>(cols, false));

    // Запускаем поиск из левого верхнего угла (0, 0).
    if (solveMaze(maze, 0, 0, path, visited)) {
        cout << "Путь найден:\n";
        // path — это список клеток от старта до финиша.
        for (auto& cell : path) {
            cout << "(" << cell.first << ", " << cell.second << ") ";
        }
        cout << '\n';
    }
    else {
        cout << "Пути нет.\n";
    }

    return 0;
            }
```

Здесь path.pop_back() и сброс visited[x][y] — это и есть «возврат»: мы отменяем выбор, который привёл в тупик.

1.9. Сравнение рекурсивных и итеративных реализаций

Факториал.	**Числа Фибоначчи.**

// Рекурсивно: O(n) времени, O(n) памяти на стек
long long factRec(int n) {
  return (n == 0) ? 1LL : n * factRec(n - 1);
}
 	
// Итеративно: O(n) времени, O(1) памяти
long long factIter(int n) {
  long long result = 1;
  for (int i = 2; i <= n; ++i) result *= i;
  return result;
}
	// Наивная рекурсия: O(2ⁿ) времени — катастрофически медленно
	long long fibRec(int n) {
	  if (n < 2) return n;
	  return fibRec(n - 1) + fibRec(n - 2);
	}
	
	// Итеративно: O(n) времени, O(1) памяти
	long long fibIter(int n) {
	  long long a = 0, b = 1;
	  for (int i = 0; i < n; ++i) {
	  long long t = a + b;
	  a = b;
	  b = t;
	  }
	  return a;
	}

Наивная рекурсия для Фибоначчи многократно пересчитывает одни и те же значения. Решения: **мемоизация** (кэш результатов) или итеративный подход.

Мемоизация (memoization) - это приём оптимизации, при котором результаты уже вычисленных вызовов функции **сохраняются** (кэшируются), и при повторном вызове с теми же аргументами результат берётся **из кэша**, а не вычисляется заново.

Сравнительная таблица.

Критерий	**Рекурсия**	**Итерация**
Краткость кода	Часто короче и ближе к математическому описанию	Обычно длиннее
Память	O(глубины) на стек вызовов	Как правило O(1)
Скорость	Есть накладные расходы на вызовы	Обычно быстрее
Риск	Переполнение стека при большой глубине	Нет такого риска

Критерий	Рекурсия	Итерация
Естественность	Идеальна для деревьев, графов, «разделяй и властвуй»	Идеальна для линейных проходов

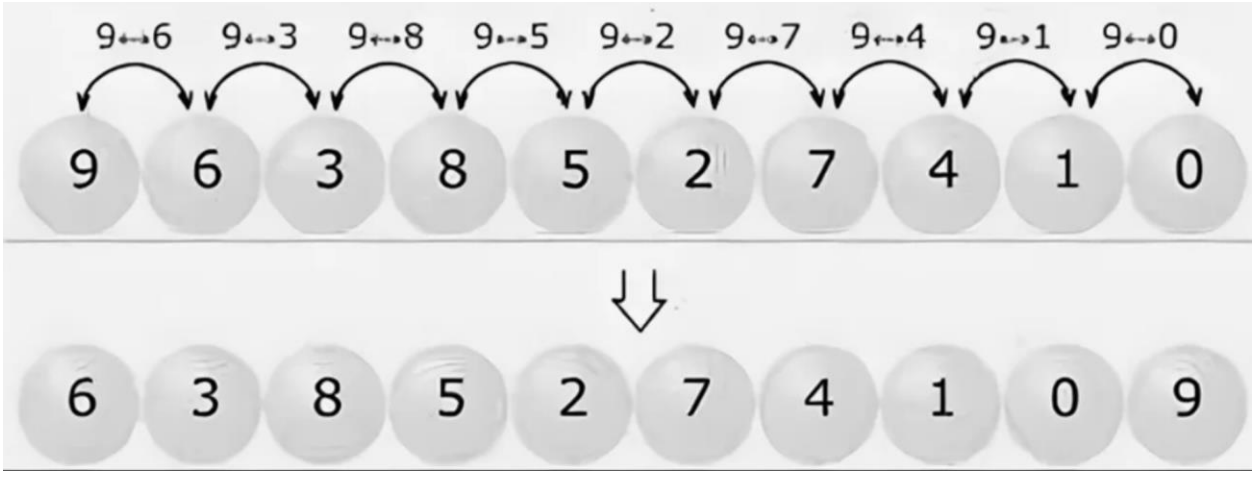
Вывод: рекурсия выбирается там, где структура задачи сама рекурсивна (деревья, графы, перебор с возвратом). Итерация - когда важны производительность и экономия памяти.

Вопрос 2. Простые сортировки

Все три рассматриваемые сортировки имеют сложность **O(n²)** в худшем и среднем случаях, работают «на месте» (in-place, O(1) дополнительной памяти) и легко реализуются.

2.1. Пузырьковая сортировка (Bubble Sort)

Идея. Многократно проходим по массиву и меняем местами соседние элементы, если они стоят в неправильном порядке. Большие элементы «всплывают» к концу.




```
void bubbleSort(vector<int>& a) {
    size_t n = a.size();
    for (size_t i = 0; i + 1 < n; ++i) {
        bool swapped = false;
        for (size_t j = 0; j + 1 < n - i; ++j) {
```

```
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;   // оптимизация: массив уже отсортирован
    }
            }
```

Сложность:

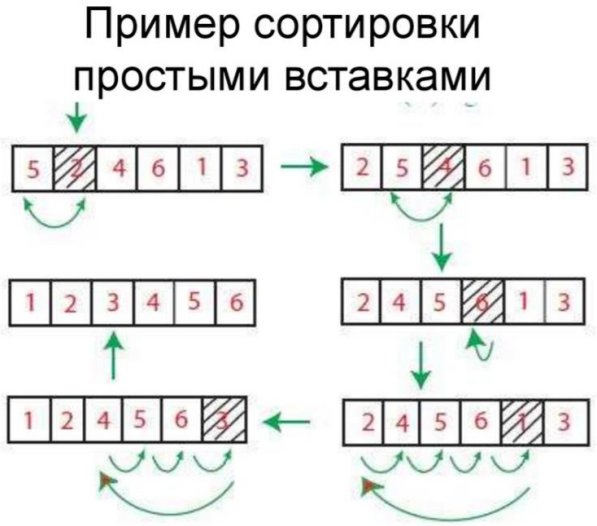
Худший случай: $O(n^2)$

Средний: $O(n^2)$

Лучший (уже отсортированный массив с флагом swapped): $O(n)$

2.2. Сортировка вставками (Insertion Sort)

Идея. Идём слева направо; каждый новый элемент «вставляем» в уже отсортированную часть массива, сдвигая большие элементы вправо.




```
void insertionSort(vector<int>& a) {
    for (size_t i = 1; i < a.size(); ++i) {
        int key = a[i];
        int j = static_cast<int>(i) - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = key;
    }
            }
```

Сложность:

1. Худший случай (обратный порядок): $O(n^2)$
2. Средний: $O(n^2)$

3. Лучший (почти отсортированный): **O(n)**

Сортировка вставками эффективна на **малых** и **почти отсортированных** массивах; часто используется как «доработка» внутри быстрой сортировки для маленьких подмассивов.

2.3. Сортировка выбором (Selection Sort)

Идея. На каждом шаге находим минимальный элемент в неотсортированной части и ставим его в начало этой части.




```
void selectionSort(vector<int>& a) {
   size_t n = a.size();
   for (size_t i = 0; i + 1 < n; ++i) {
      size_t minIdx = i;
      for (size_t j = i + 1; j < n; ++j) {
         if (a[j] < a[minIdx]) minIdx = j;
      }
      if (minIdx != i) swap(a[i], a[minIdx]);
   }
           }
```

Сложность:

Все случаи: **O(n²)** - число сравнений всегда n(n−1)/2, независимо от исходного порядка.

Число обменов: O(n) - минимум среди трёх сортировок.

2.4. Анализ сложности O(n²)

Для всех трёх сортировок число сравнений в худшем случае:

$(n-1) + (n-2) + ... + 1 = n(n-1)/2 = O(n^2)$

Почему это важно:

1. При n = 1 000 → ~500 000 операций (приемлемо);
2. При n = 1 000 000 → $\sim 5 \cdot 10^{11}$ операций (неприемлемо).
3. Поэтому на больших данных применяют O(n log n)-сортировки (быстрая, слиянием, пирамидальная).
4.

2.5. Устойчивость и адаптивность

Устойчивость. Сортировка **устойчива**, если равные элементы сохраняют свой относительный порядок.

Сортировка	Устойчива?	Почему
Пузырьковая	Да	Обмен только при строгом a[j] > a[j+1]
Вставками	Да	Вставка происходит после всех равных элементов
Выбором	Нет	Обмен может «перепрыгнуть» равный элемент

Пример неустойчивости сортировки выбором: массив [2a, 2b, 1] → после первого шага [1, 2b, 2a] - порядок 2a и 2b изменился.

Адаптивность. Сортировка **адаптивна**, если её время работы сокращается на почти отсортированных данных.

Сортировка	Адаптивна?	Лучший случай
Пузырьковая (с флагом)	Да	O(n)

Сортировка	Адаптивна?	Лучший случай
Вставками	Да	O(n)
Выбором	✕ Нет	$O(n^2)$

Вывод. Если важна устойчивость - выбираем пузырьковую или вставками. Если данные почти отсортированы - вставками (самая эффективная из трёх на практике). Если критично число обменов - выбором.

Итоги лекции

Рекурсия - метод решения задач через сведение к подзадачам меньшего размера. Обязательны базовый случай и шаг рекурсии.

Различают **прямую** и **косвенную** рекурсию.

Рекурсия естественна для **списков, деревьев, перебора с возвратом** (Ханойские башни, лабиринт).

Рекурсивные реализации часто короче, но проигрывают итеративным по памяти и скорости; наивная рекурсия для Фибоначчи имеет сложность $O(2^n)$.

Простые сортировки (пузырьковая, вставками, выбором) имеют сложность $O(n^2)$, работают in-place.

Устойчивость: пузырьковая и вставками - да, выбором - нет.

Адаптивность: пузырьковая (с флагом) и вставками - да, выбором - нет.